

Графовые нейронные сети

- **Encoder:** Maps each node to a low-dimensional vector

$$\text{ENC}(v) = \mathbf{z}_v$$

node in the input graph

d -dimensional embedding

- **Similarity function:** Specifies how the relationships in vector space map to the relationships in the original network

$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

Similarity of u and v in the original network

dot product between node embeddings

Decoder

- **Limitations** of shallow embedding methods:
 - **$O(|V|)$ parameters are needed:**
 - No sharing of parameters between nodes
 - Every node has its own unique embedding
 - **Inherently “transductive”:**
 - Cannot generate embeddings for nodes that are not seen during training
 - **Do not incorporate node features:**
 - Nodes in many graphs have features that we can and should leverage

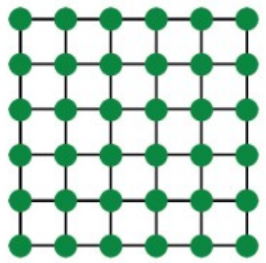
- **Today:** We will now discuss deep learning methods based on **graph neural networks (GNNs)**:

$$\text{ENC}(v) = \begin{array}{l} \text{multiple layers of} \\ \text{non-linear transformations} \\ \text{based on graph structure} \end{array}$$

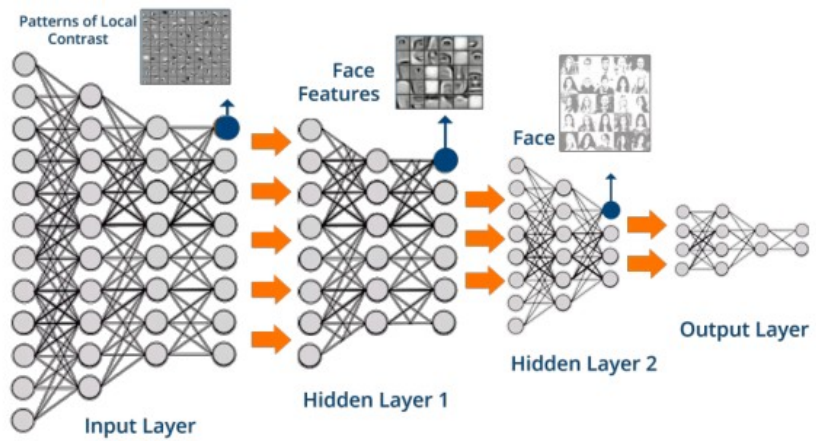
- **Note:** All these deep encoders can be **combined with node similarity functions** defined in the Lecture 3.

Tasks we will be able to solve:

- Node classification
 - Predict a type of a given node
- Link prediction
 - Predict whether two nodes are linked
- Community detection
 - Identify densely linked clusters of nodes
- Network similarity
 - How similar are two (sub)networks

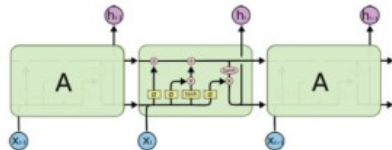


Images



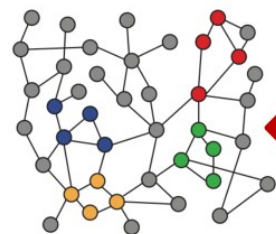
Сеточный граф (grid graph)

Text/Speech

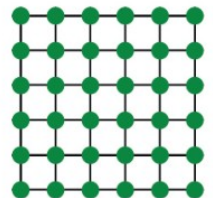


Цепной граф (chain graph)

- Arbitrary size and complex topological structure (i.e., no spatial locality like grids)



Networks



Images

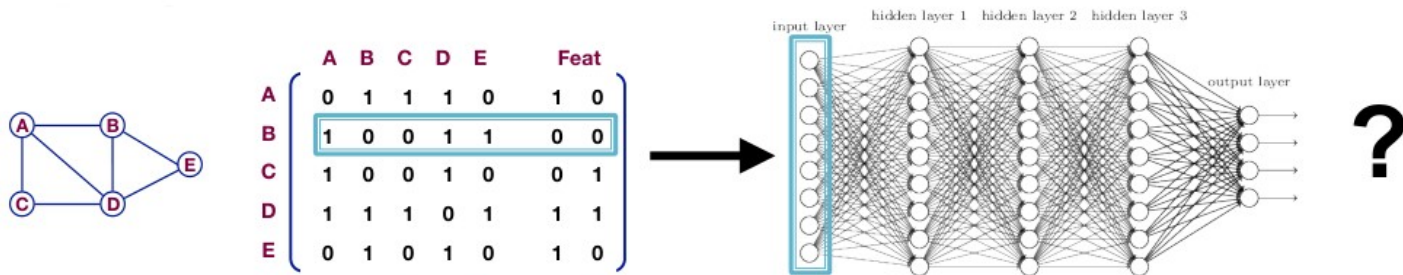


Text

- No fixed node ordering or reference point
- Often dynamic and have multimodal features

- **Assume we have a graph G :**
 - V is the **vertex set**
 - A is the **adjacency matrix** (assume binary)
 - $X \in \mathbb{R}^{m \times |V|}$ is a matrix of **node features**
 - v : a node in V ; $N(v)$: the set of neighbors of v .
 - **Node features:**
 - Social networks: User profile, User image
 - Biological networks: Gene expression profiles, gene functional information
 - When there is no node feature in the graph dataset:
 - Indicator vectors (one-hot encoding of a node)
 - Vector of constant 1: $[1, 1, \dots, 1]$

- Join adjacency matrix and features
- Feed them into a deep neural net:



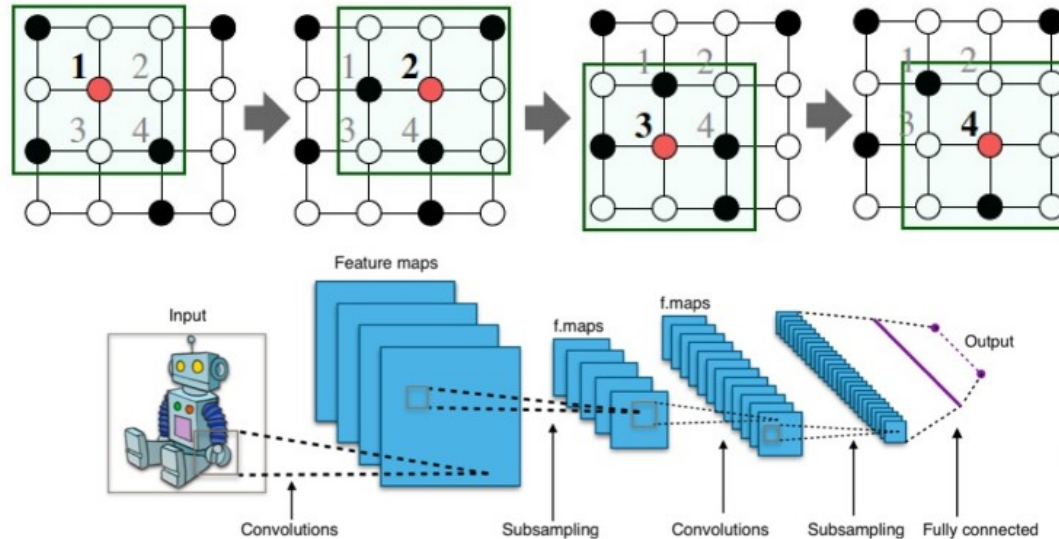
■ Issues with this idea:

- $O(|V|)$ parameters
- Not applicable to graphs of different sizes
- Sensitive to node ordering

* Параметров всегда больше чем нод...

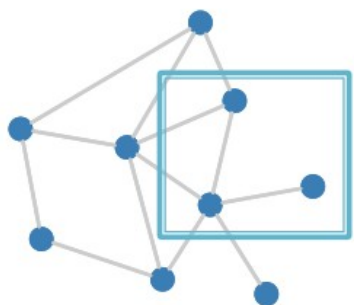
* Один граф может быть представлен множеством матриц смежности...

CNN on an image:

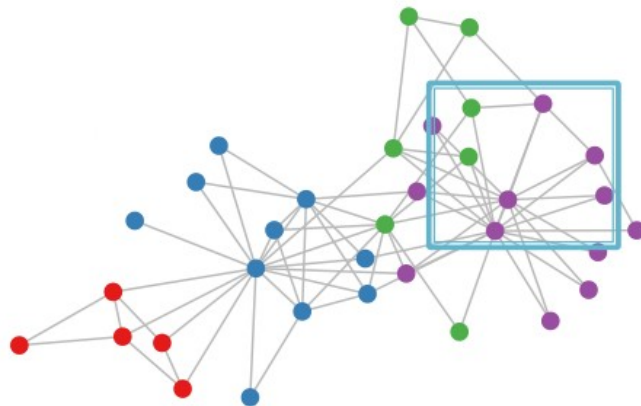


Goal is to generalize convolutions beyond simple lattices
Leverage node features/attributes (e.g., text, images)

But our graphs look like this:



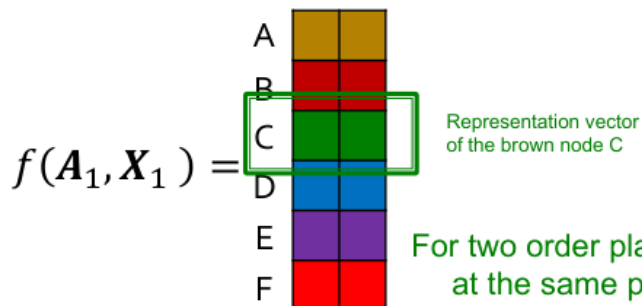
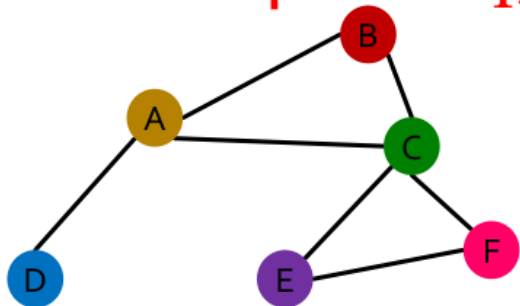
or this:



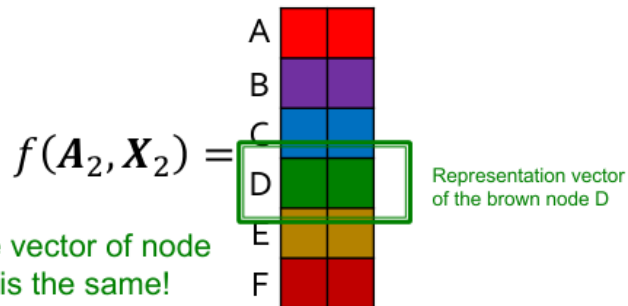
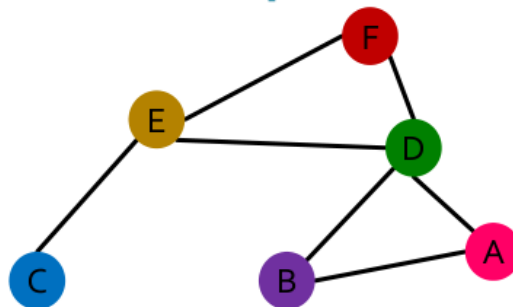
- There is no fixed notion of locality or sliding window on the graph
- Graph is permutation invariant

Similarly for node representation: We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{m \times d}$.

Order plan 1: A_1, X_1



Order plan 2: A_2, X_2

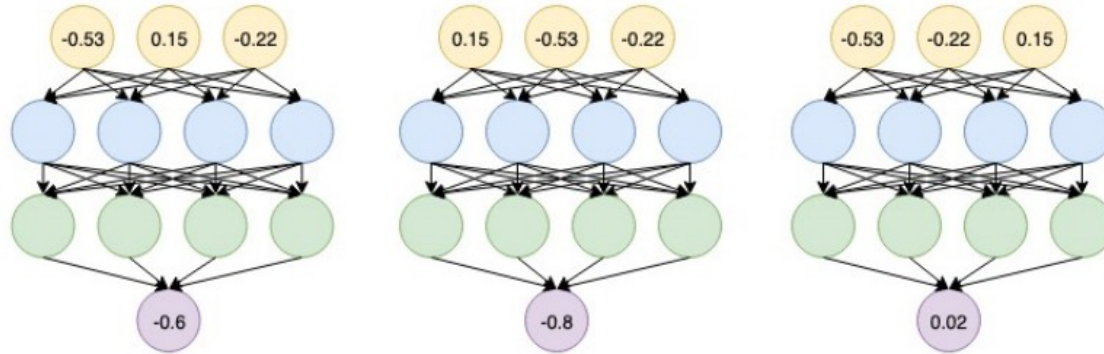


For two order plans, the vector of node at the same position is the same!

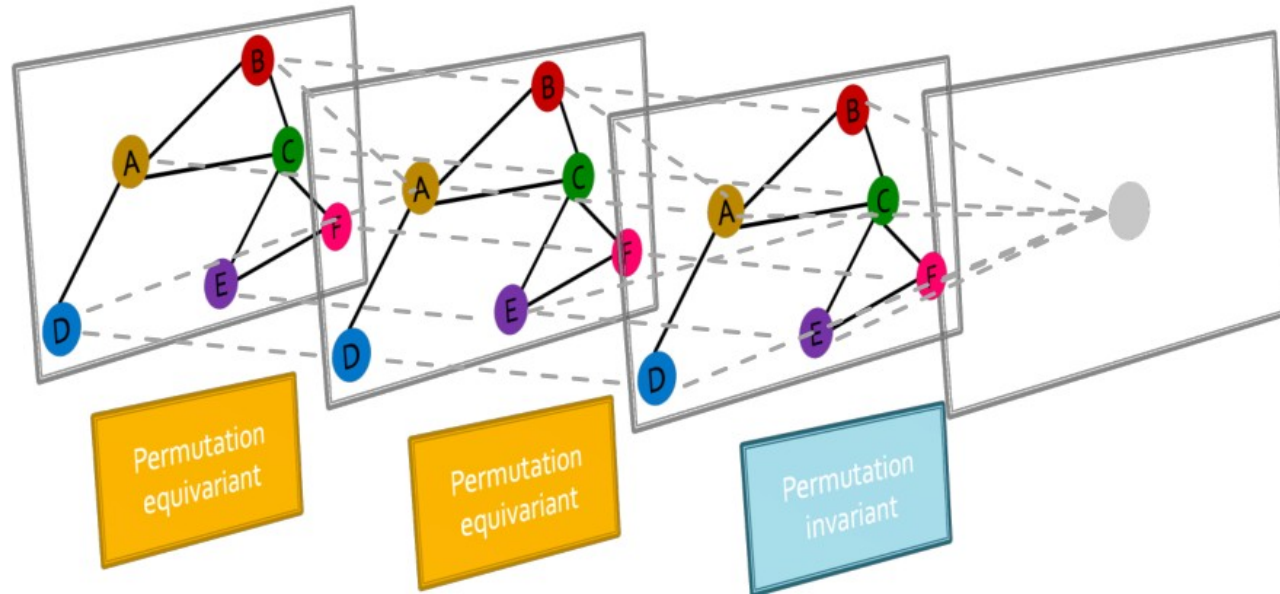
Are other neural network architectures, e.g., MLPs, permutation invariant / equivariant?

■ No.

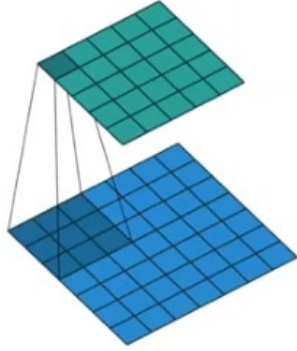
Switching the order of the input leads to different outputs!



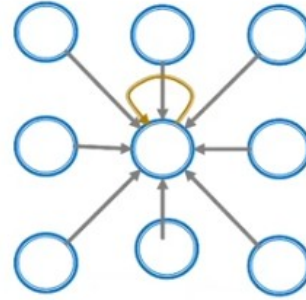
- Graph neural networks consist of multiple permutation equivariant / invariant functions.



Single Convolutional neural network (CNN) layer with 3x3 filter:



Image

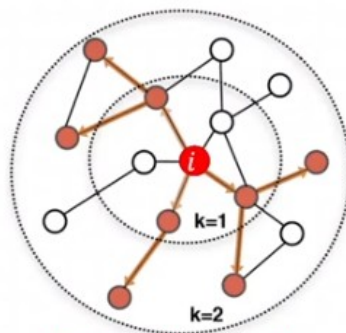


Graph

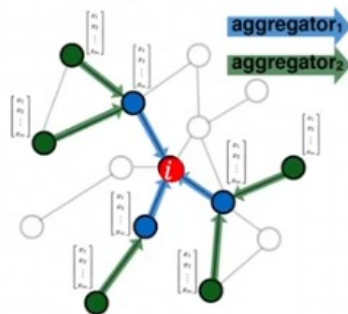
Idea: transform information at the neighbors and combine it:

- Transform “messages” h_i from neighbors: $W_i h_i$
- Add them up: $\sum_i W_i h_i$

Idea: Node's neighborhood defines a computation graph



Determine node
computation graph

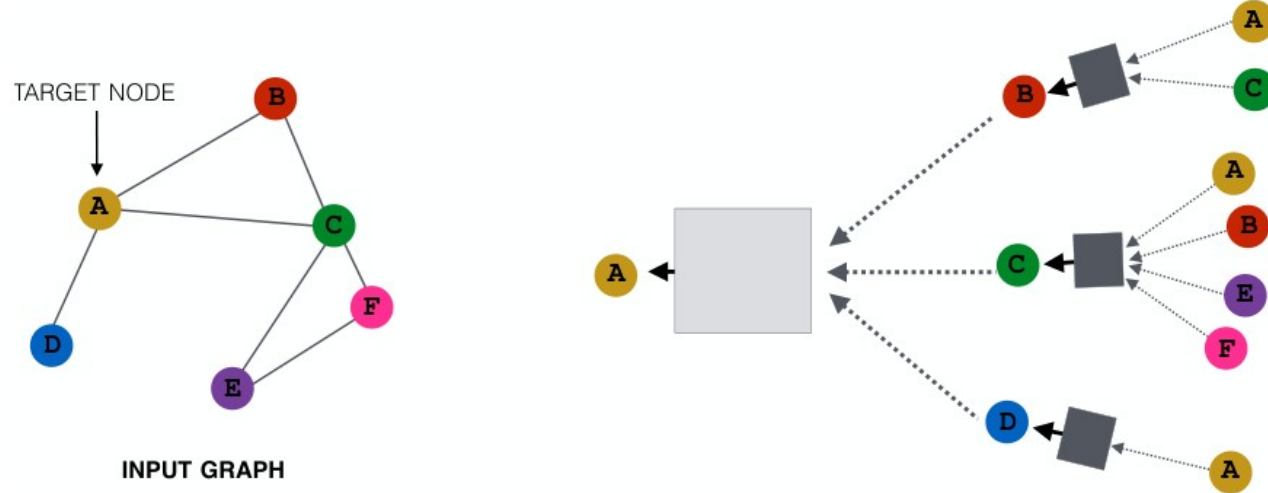


Propagate and
transform information

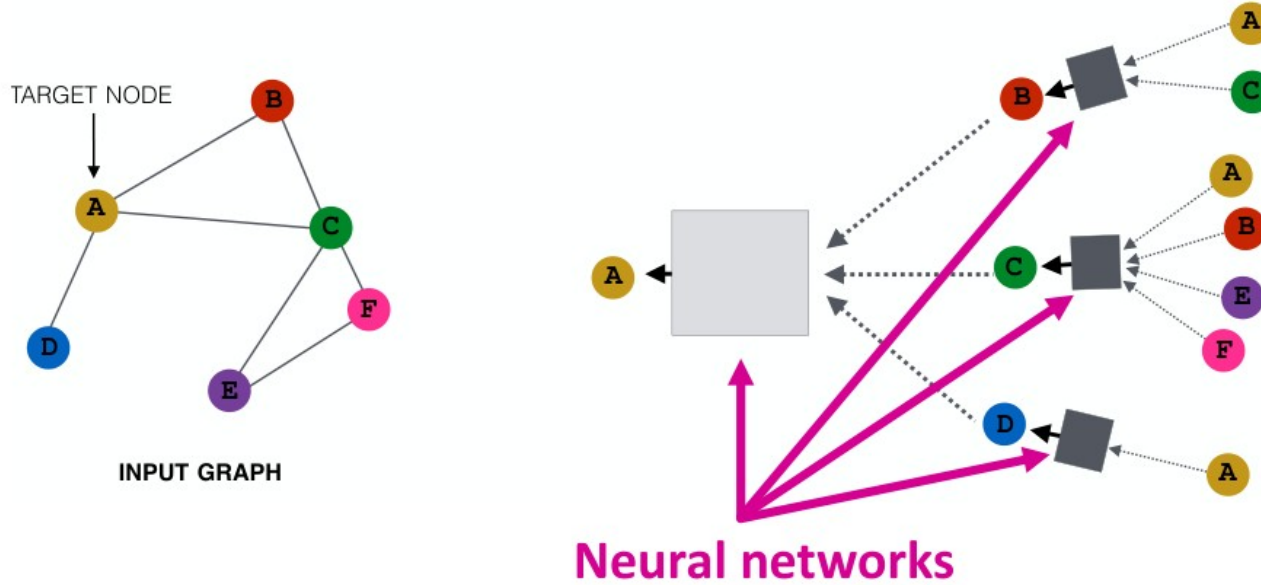
Learn how to propagate information across the graph to compute node features

Вычислительный граф определяет топологию графа!

- **Key idea:** Generate node embeddings based on **local network neighborhoods**

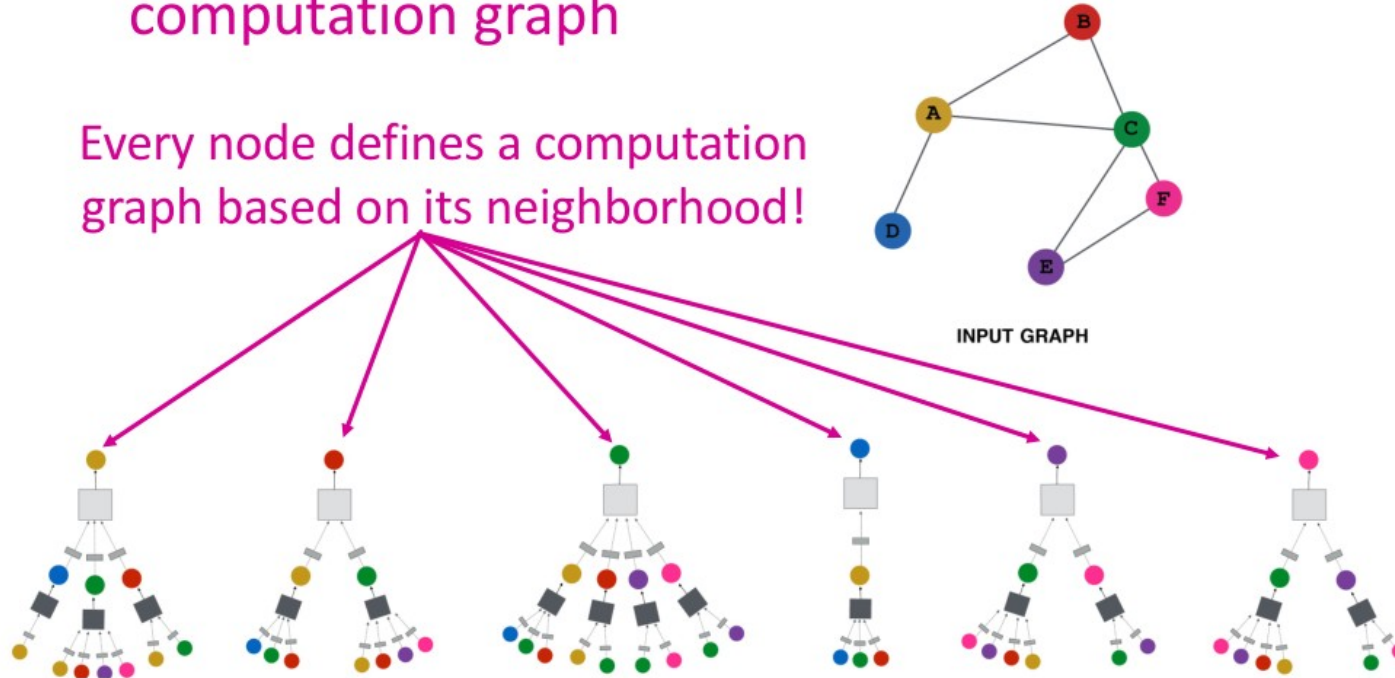


- **Intuition:** Nodes aggregate information from their neighbors using neural networks



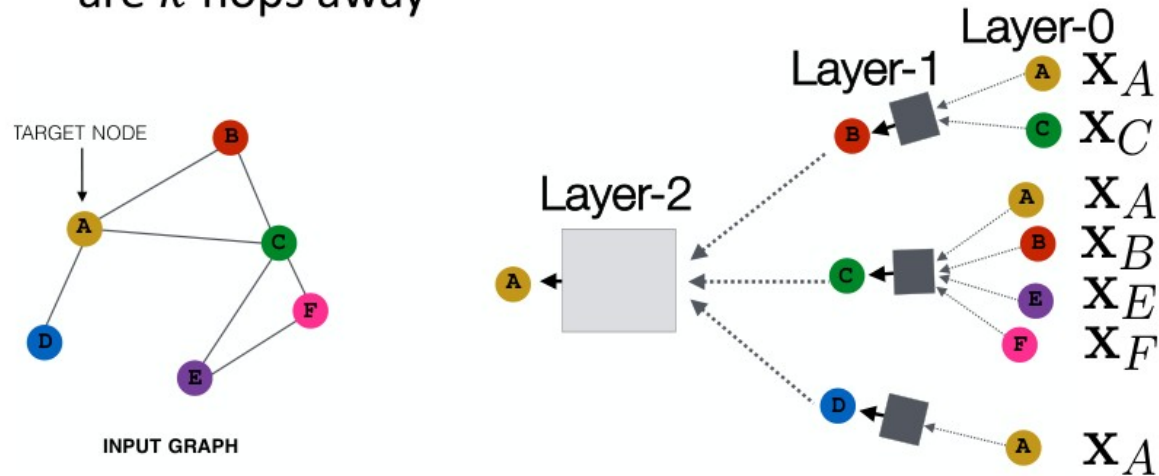
- **Intuition:** Network neighborhood defines a computation graph

Every node defines a computation graph based on its neighborhood!



У каждого нода – своя нейронная сеть собственной архитектуры

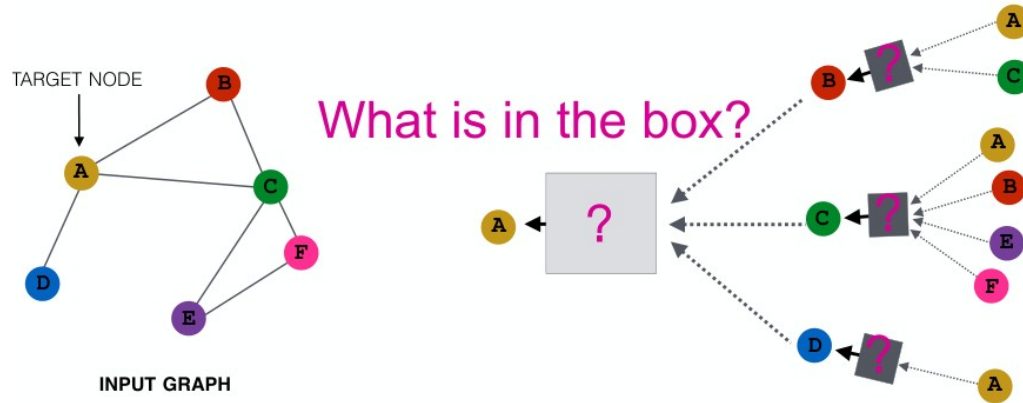
- Model can be of arbitrary depth:
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node v is its input feature, x_v
 - Layer- k embedding gets information from nodes that are k hops away



На каждом слое эмбединги одной и той-же ноды могут быть различными!

Агрегация соседей

- **Neighborhood aggregation:** Key distinctions are in how different approaches aggregate information across the layers



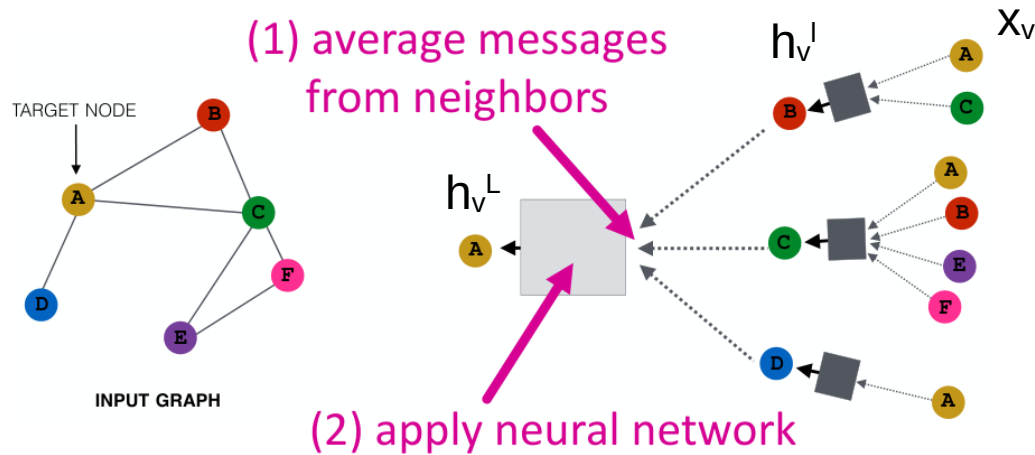
Агрегация соседей

Метод агрегации должен быть инвариантен к перестановкам:

$$a+b = b+a$$

$$a*b = b*a$$

- **Basic approach:** Average information from neighbors and apply a neural network



The mathematical formulation of the node aggregation process is given by the following equation:

$$h_v^{(k+1)} = \sigma \left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)} \right), \forall k \in \{0, \dots, K-1\}$$

Where:

- $h_v^0 = x_v$: Initial 0-th layer embeddings are equal to node features.
- $h_v^{(k)}$: embedding of v at layer k .
- σ : Non-linearity (e.g., ReLU).
- W_k : Weight matrix for the aggregation step.
- B_k : Bias matrix for the aggregation step.
- $N(v)$: Set of neighbors of node v .
- $|N(v)|$: Number of neighbors of node v .
- $z_v = h_v^{(K)}$: Embedding after L layers of neighborhood aggregation.
- K : Total number of layers.

Notice summation is a permutation invariant pooling/aggregation.

Агрегация соседей

Trainable weight matrices
(i.e., what we learn)

$$h_v^{(0)} = x_v$$
$$h_v^{(k+1)} = \sigma \left(\boxed{W_k} \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + \boxed{B_k} h_v^{(k)} \right), \forall k \in \{0..K-1\}$$
$$z_v = h_v^{(K)}$$

Final node embedding

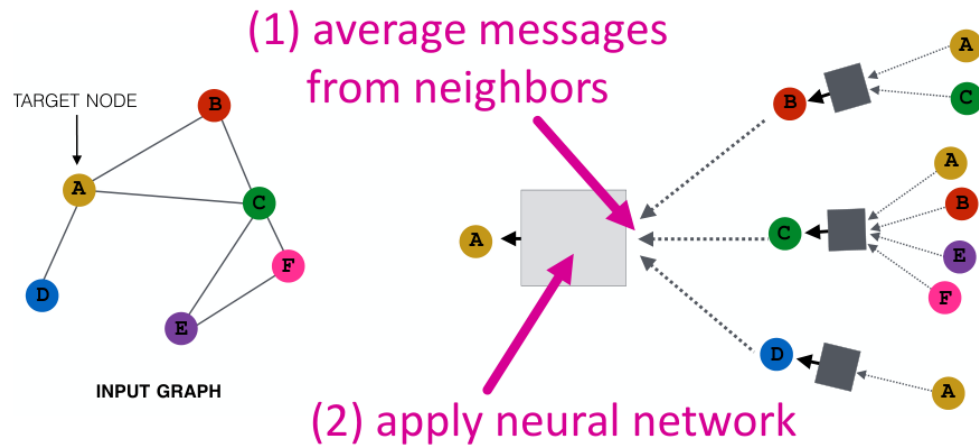
We can feed these **embeddings into any loss function**
and run SGD to **train the weight parameters**

h_v^k : the hidden representation of node v at layer k

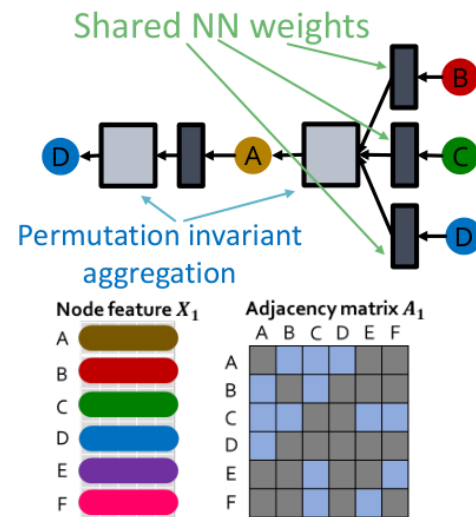
- W_k : weight matrix for neighborhood aggregation
- B_k : weight matrix for transforming hidden vector of self

Агрегация соседей

- **Basic approach:** Average information from neighbors and apply a neural network



Message passing and neighbor aggregation in graph convolution networks is permutation equivariant.



Матричная формулировка задачи

- **Many aggregations can be performed efficiently by (sparse) matrix operations**

- Let $H^{(k)} = [h_1^{(k)} \dots h_{|V|}^{(k)}]^T$

Matrix of hidden embeddings $H^{(k-1)}$

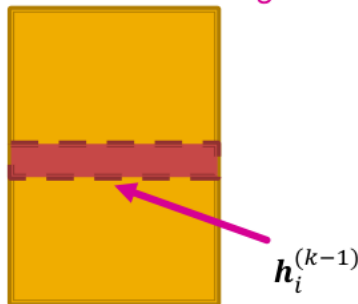
- Then: $\sum_{u \in N_v} h_u^{(k)} = A_{v,:} H^{(k)}$

- Let D be diagonal matrix where $D_{v,v} = \text{Deg}(v) = |N(v)|$

- The inverse of D : D^{-1} is also diagonal:
 $D_{v,v}^{-1} = 1/|N(v)|$

- **Therefore,**

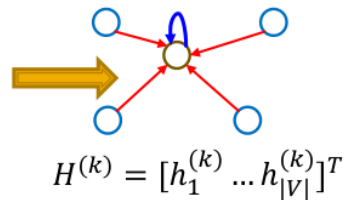
$$\sum_{u \in N(v)} \frac{h_u^{(k-1)}}{|N(v)|} \Rightarrow H^{(k+1)} = D^{-1} A H^{(k)}$$



- Re-writing update function in matrix form:

$$H^{(k+1)} = \sigma(\tilde{A} H^{(k)} W_k^T + H^{(k)} B_k^T)$$

where $\tilde{A} = D^{-1} A$



- Red: neighborhood aggregation

- Blue: self transformation

- In practice, this implies that efficient sparse matrix multiplication can be used ($\tilde{A}$ is sparse)

- Node embedding $\mathbf{z}_v$ is a function of input graph
- **Supervised setting:** we want to minimize the loss $\mathcal{L}$ (see also Slide 15):

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{z}_v))$$

- $\mathbf{y}$: node label
- $\mathcal{L}$ could be L2 if $\mathbf{y}$ is real number, or cross entropy if $\mathbf{y}$ is categorical
- **Unsupervised setting:**
 - No node label available
 - **Use the graph structure as the supervision!**
i.e. maximize $\mathbf{z}_u^T \mathbf{z}_v$ for 'similar' nodes

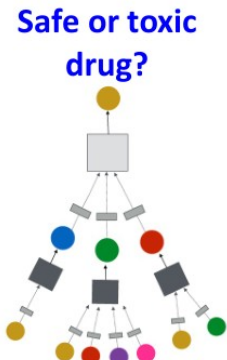
- “Similar” nodes have similar embeddings

$$\mathcal{L} = \sum_{z_u, z_v} \text{CE}(y_{u,v}, \text{DEC}(z_u, z_v))$$

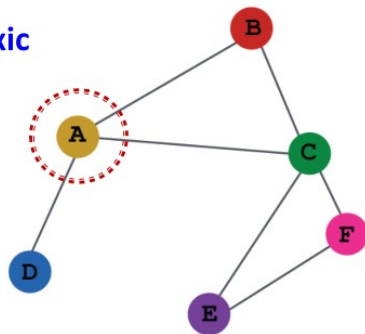
- Where $y_{u,v} = 1$ when node u and v are similar
- **CE** is the cross entropy (Slide 16)
- **DEC** is the decoder such as inner product (Lecture 4)
- **Node similarity** can be anything from Lecture 3, e.g., a loss based on:
 - **Random walks** (node2vec, DeepWalk, struc2vec)
 - **Matrix factorization**
 - **Node proximity in the graph**

Обучение сети с учителем

Directly train the model for a supervised task
(e.g., node classification)



Safe or toxic
drug?



E.g., a drug-drug
interaction network

Directly train the model for a supervised task
(e.g., **node classification**)

- Use cross entropy loss (Slide 16)

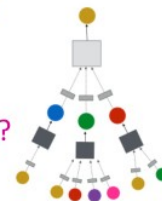
$$\mathcal{L} = \sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v) \log(1 - \sigma(z_v^T \theta))$$

Encoder output:
node embedding

Classification
weights

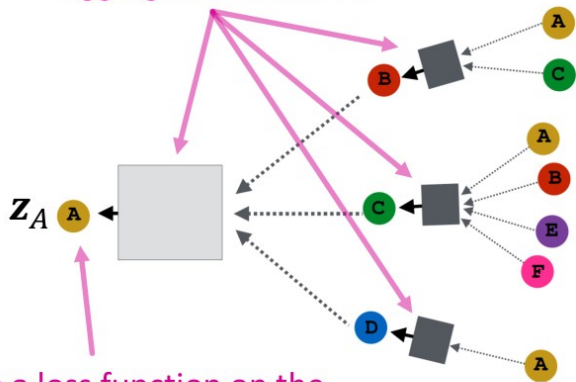
Node class
label

Safe or toxic drug?

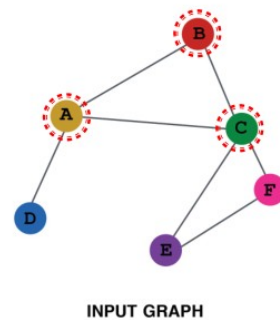


Обучение сети с учителем

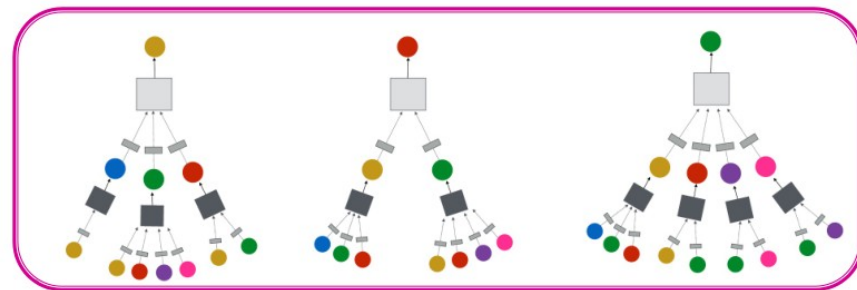
(1) Define a neighborhood aggregation function



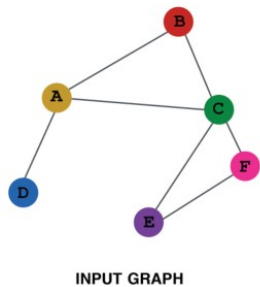
(2) Define a loss function on the embeddings



(3) Train on a set of nodes, i.e., a batch of compute graphs

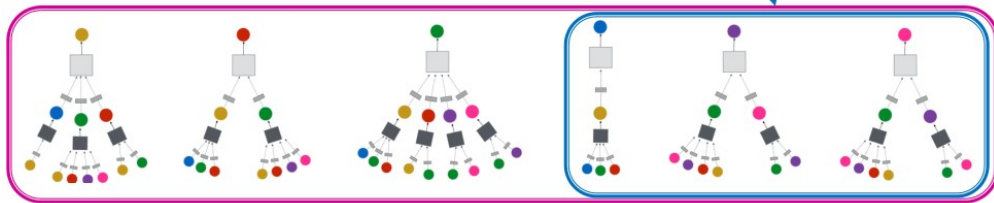


Обучение сети с учителем

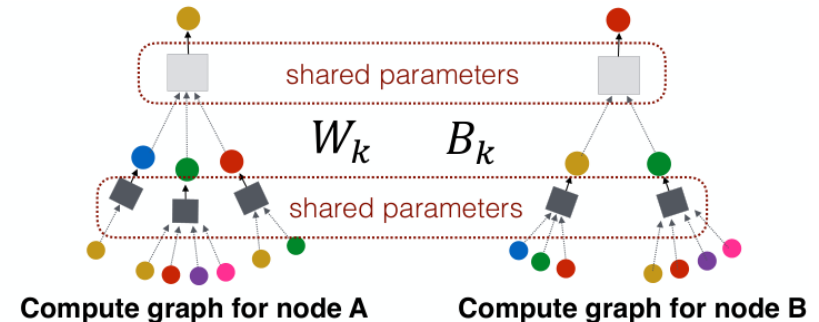
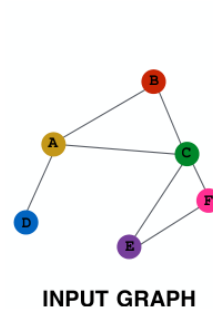


(4) Generate embeddings
for nodes as needed

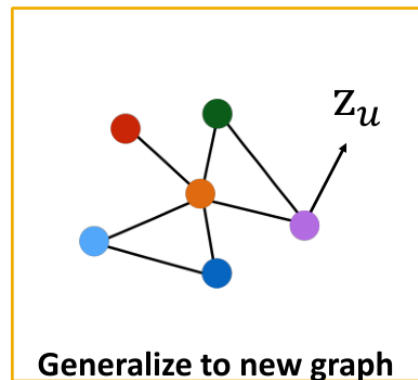
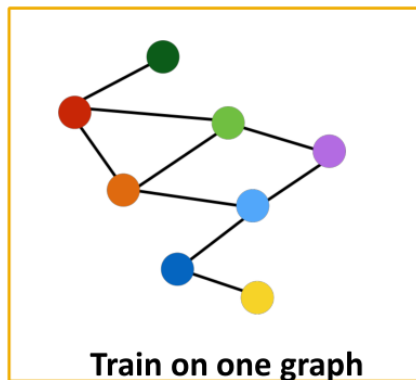
Even for nodes we never
trained on!



- The number of model parameters is sublinear in $|V|$ and we can **generalize to unseen nodes**!

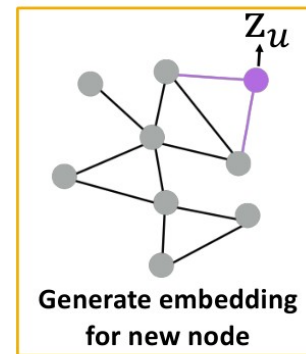
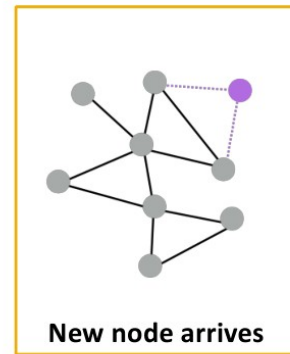
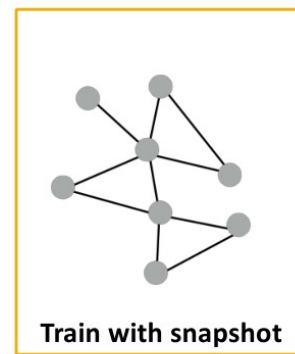


Обобщающие возможности сети



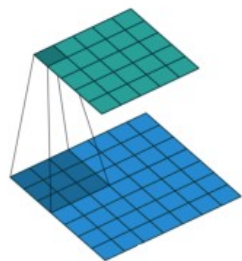
Inductive node embedding → Generalize to entirely unseen graphs

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

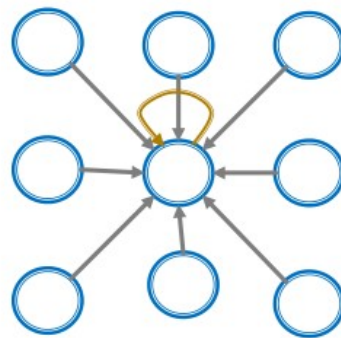


- Many application settings constantly encounter previously unseen nodes:
 - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

Convolutional neural network (CNN) layer with 3x3 filter:



Image



Graph

$$\text{GNN formulation: } h_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$$

$$\text{CNN formulation: } h_v^{(l+1)} = \sigma(\sum_{u \in N(v)} \mathbf{W}_l^u h_u^{(l)} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$$

Key difference: We can learn different $\mathbf{W}_l^u$ for different “neighbor” u for pixel v on the image. The reason is we can pick an order for the 9 neighbors using **relative position** to the center pixel: $\{(-1, -1), (-1, 0), (-1, 1), \dots, (1, 1)\}$

GCNN

(Graph convolutional neural network)

$$h_i = \frac{1}{\deg(i)} \sum_{j \in \mathcal{N}_i} x_j W^T$$

```
D = np.array([ [3, 0, 0, 0], [0, 1, 0, 0], [0, 0, 2, 0], [0, 0, 0, 2] ])
print(D)
```

$$H = \tilde{A}^T X W^T \quad \tilde{A} = A + I \text{ .}$$

```
D_inv1=np.linalg.inv(D + np.identity(4))
print(D_inv1)
```

$$\tilde{D}^{-1} = (D + I)^{-1}$$

$\tilde{D}^{-1} \tilde{A} X W^T$ will normalize every row of features.

$\tilde{A} \tilde{D}^{-1} X W^T$ will normalize every column of features.

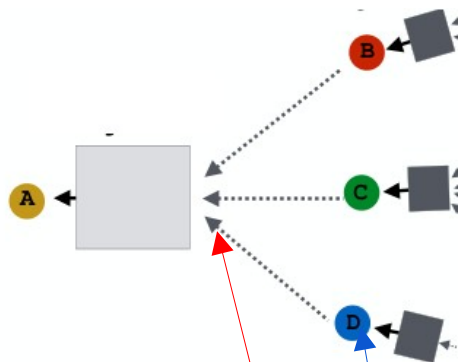
Как выбрать, как нормализовать?

Реализация:

$$H = \tilde{D}^{-\frac{1}{2}} \tilde{A}^T \tilde{D}^{-\frac{1}{2}} X W^T$$

$$h_i = \sum_{j \in \mathcal{N}_i} \frac{1}{\sqrt{\deg(i)} \sqrt{\deg(j)}} x_j W^T$$

GCNconv layer

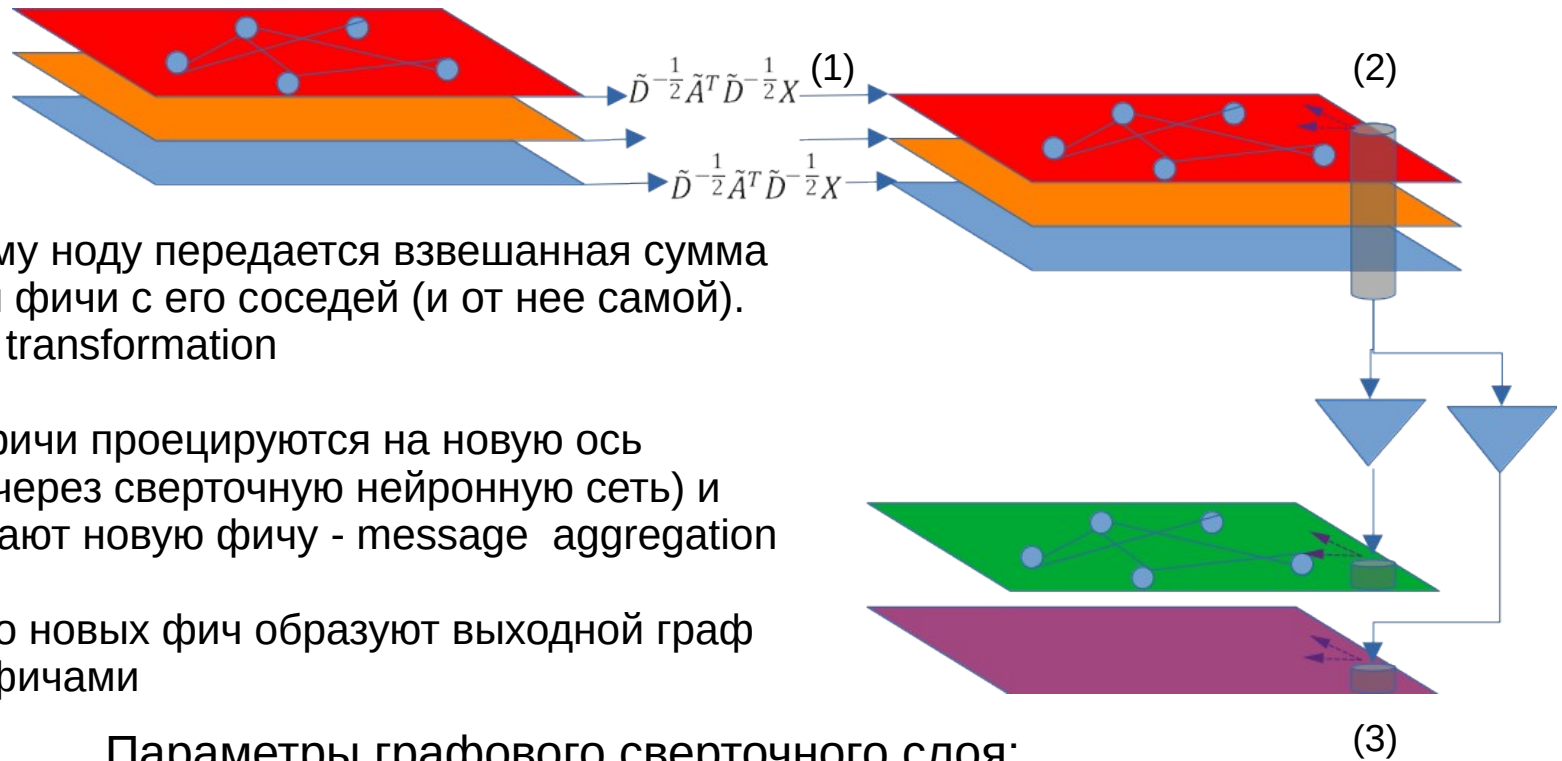


$$\mathbf{X}' = \hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \mathbf{X} \Theta,$$

Матрица преобразования признаков

- **input:** node features $(|\mathcal{V}|, F_{in})$, edge indices $(2, |\mathcal{E}|)$ or sparse matrix $(|\mathcal{V}|, |\mathcal{V}|)$, edge weights $(|\mathcal{E}|)$ (*optional*)
- **output:** node features $(|\mathcal{V}|, F_{out})$

GCNconv layer



1. К каждому ноду передается взвешанная сумма выбранной фичи с его соседей (и от нее самой).
- message transformation

2. Новые фичи проецируются на новую ось (проходят через сверточную нейронную сеть) и образуют новую фичу - message aggregation

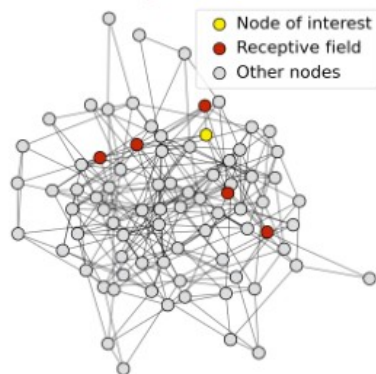
3. Несколько новых фич образуют выходной граф с новыми фичами

Параметры графового сверточного слоя:

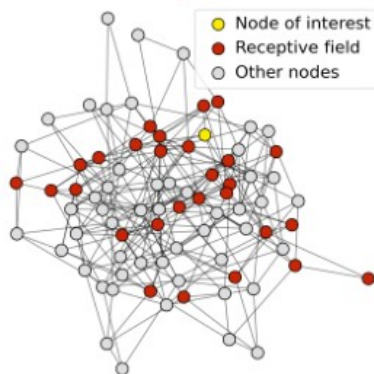
- **input:** node features $(|\mathcal{V}|, F_{in})$, edge indices $(2, |\mathcal{E}|)$ or sparse matrix $(|\mathcal{V}|, |\mathcal{V}|)$, edge weights $(|\mathcal{E}|)$ (optional)
- **output:** node features $(|\mathcal{V}|, F_{out})$

- **Receptive field:** the set of nodes that determine the embedding of a node of interest
 - In a K -layer GNN, each node has a receptive field of K -hop neighborhood

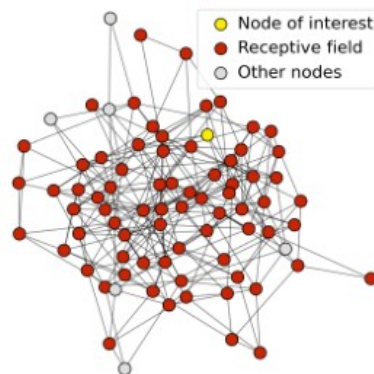
Receptive field for
1-layer GNN



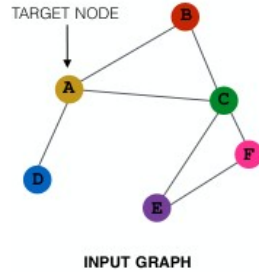
Receptive field for
2-layer GNN



Receptive field for
3-layer GNN



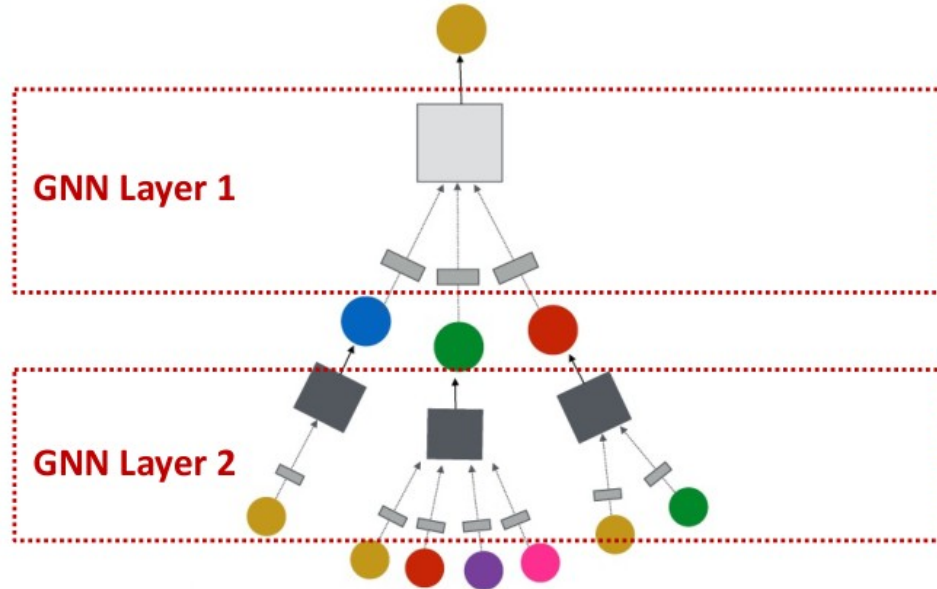
Последовательное включение графовых сверточных слоев (stacking)



How to connect GNN layers into a GNN?

- Stack layers sequentially
- Ways of adding skip connections

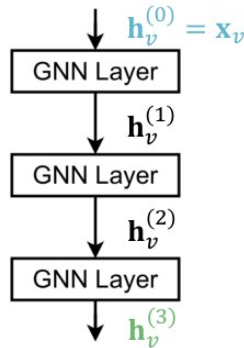
(3) Layer connectivity



Последовательное включение графовых сверточных слоев (stacking)

■ How to construct a Graph Neural Network?

- The standard way: Stack GNN layers sequentially
- Input: Initial raw node feature $\mathbf{x}_v$
- Output: Node embeddings $\mathbf{h}_v^{(L)}$ after L GNN layers



■ The Issue of stacking many GNN layers

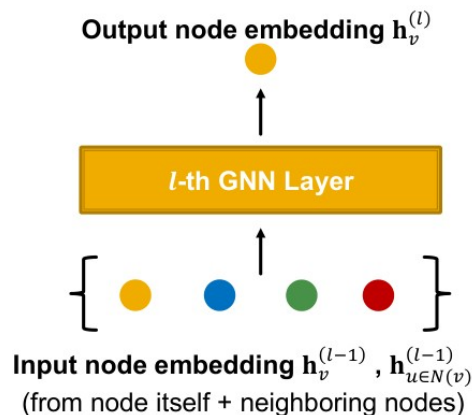
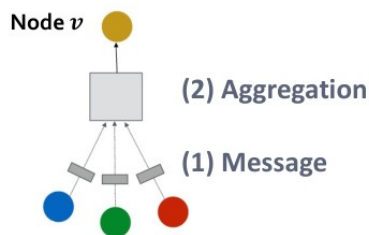
- GNN suffers from **the over-smoothing problem**
- **The over-smoothing problem: all the node embeddings converge to the same value**
 - This is bad because we **want to use node embeddings to differentiate nodes**
- Why does the over-smoothing problem happen?

■ Idea of a GNN Layer:

- Compress a set of vectors into a single vector

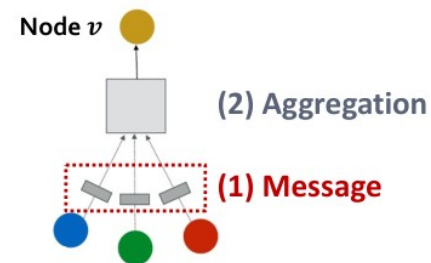
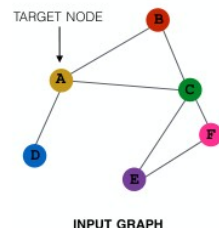
▪ Two-step process:

- (1) Message
- (2) Aggregation



■ (1) Message computation

- **Message function:** $\mathbf{m}_u^{(l)} = \text{MSG}^{(l)}(\mathbf{h}_u^{(l-1)})$
- **Intuition:** Each node will create a message, which will be sent to other nodes later
- **Example:** A Linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$
 - Multiply node features with weight matrix $\mathbf{W}^{(l)}$



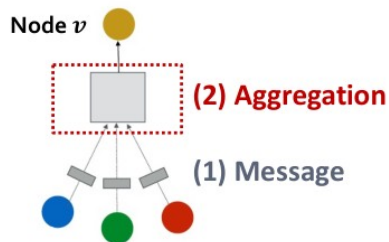
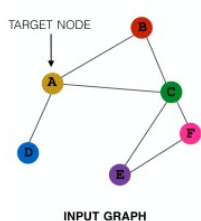
■ (2) Aggregation

- **Intuition:** Each node will aggregate the messages from node v 's neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$$

- **Example:** Sum($\cdot$), Mean($\cdot$) or Max($\cdot$) aggregator

- $\mathbf{h}_v^{(l)} = \text{Sum}(\{\mathbf{m}_u^{(l)}, u \in N(v)\})$



- **Issue:** Information from node v itself **could get lost**

- Computation of $\mathbf{h}_v^{(l)}$ does not directly depend on $\mathbf{h}_v^{(l-1)}$

- **Solution:** Include $\mathbf{h}_v^{(l-1)}$ when computing $\mathbf{h}_v^{(l)}$

- **(1) Message:** compute message from node v itself

- Usually, a **different message computation** will be performed

$$\text{Blue, Green, Red} \quad \mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \quad \text{Yellow} \quad \mathbf{m}_v^{(l)} = \mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)}$$

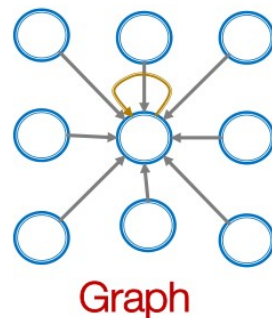
- **(2) Aggregation:** After aggregating from neighbors, we can **aggregate the message from node v itself**

- Via **concatenation** or **summation**

Then aggregate from node itself

$$\mathbf{h}_v^{(l)} = \text{CONCAT} \left(\underbrace{\text{AGG} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)}_{\text{First aggregate from neighbors}}, \underbrace{\mathbf{m}_v^{(l)}}_{\text{Then aggregate from node itself}} \right)$$

Нужно в граф добавлять петлю на сам нод!



Putting things together:

- (1) **Message**: each node computes a message

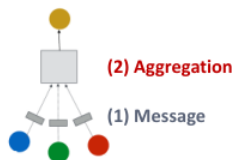
$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

- (2) **Aggregation**: aggregate messages from neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

- Nonlinearity (activation)**: Adds expressiveness

- Often written as $\sigma(\cdot)$: $\text{ReLU}(\cdot)$, $\text{Sigmoid}(\cdot)$, ...
- Can be added to **message or aggregation**



(1) Graph Convolutional Networks (GCN)

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \right)$$

Aggregation + Transformation (GraphSAGE)

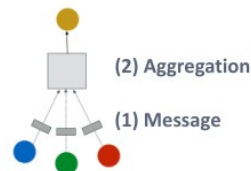
- How to write this as Message + Aggregation?

Transformation + Aggregation (GraphCNN)

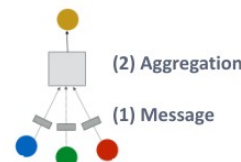
$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \boxed{\mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}} \right)$$

Message

Aggregation



$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \right)$$



- Message:**

- Each Neighbor: $\mathbf{m}_u^{(l)} = \frac{1}{|N(v)|} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$

Normalized by node degree
(In the GCN paper they use a slightly different normalization)

- Aggregation:**

- Sum** over messages from neighbors, then apply activation

$$\mathbf{h}_v^{(l)} = \sigma \left(\text{Sum} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right) \right)$$

In GCN graph is assumed to have self-edges that are included in the summation.

Graph convolution from the scratch

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
class GCNLayer(nn.Module):
    def __init__(self, input_dim: int, output_dim: int, A: torch.Tensor):
        super(GCNLayer, self).__init__()
        self.input_dim = input_dim
        self.output_dim = output_dim

        self.A = A
        self.A_hat = self.A + torch.eye(self.A.size(0))

        self.ones = torch.ones(input_dim, input_dim)
        self.D = torch.matmul(self.A.float(), self.ones.float())
        self.D = torch.diag(self.D)
        self.D = torch.diag_embed(self.D)
        self.D_neg_sqrt = torch.diag_embed(torch.diag(torch.pow(self.D, -0.5)))

        self.W = nn.Parameter(torch.rand(input_dim, output_dim))

    def forward(self, X: torch.Tensor):
        support_1 = torch.matmul(self.D_neg_sqrt, torch.matmul(self.A_hat, self.D_neg_sqrt))
        support_2 = torch.matmul(support_1, torch.matmul(X, self.W))
        H = F.relu(support_2)
        return H
```

```
input_dim = 3
output_dim = 2
A = torch.tensor([[1., 0., 0.], [0., 1., 1.], [0., 1., 1.]])
gcn_layer = GCNLayer(input_dim, output_dim, A)
X = torch.tensor([[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]])
output = gcn_layer(X)
```

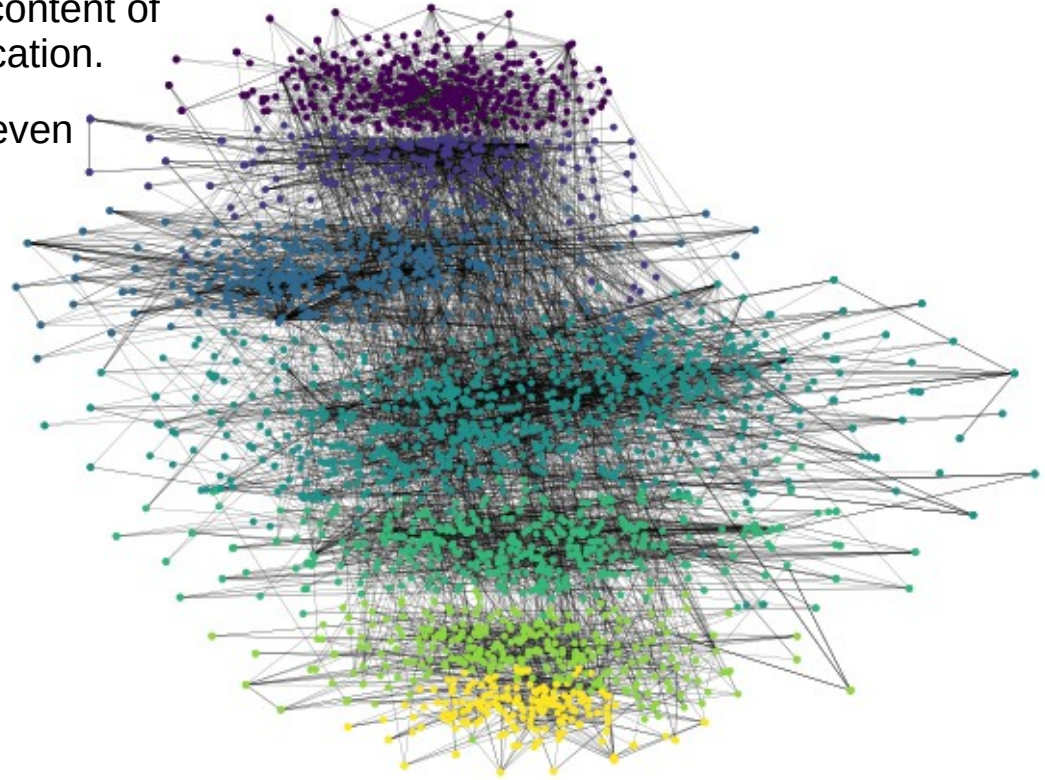
Cora dataset

Each paper in the Cora dataset is described by a binary word vector, which serves as the feature set for the dataset.

The presence (1) or absence (0) of each word from a dictionary of 1,433 unique words is recorded in this vector. This high-dimensional feature space captures the content of each paper, enabling detailed analysis and classification.

The labels in the Cora dataset correspond to the seven classes of machine learning topics:

- 1) Case Based
- 2) Genetic Algorithms
- 3) Neural Networks
- 4) Probabilistic Methods
- 5) Reinforcement Learning
- 6) Rule Learning
- 7) Theory



```

import torch
import torch.nn.functional as F
from torch_geometric.nn import GCNConv

def accuracy(pred_y, y):
    return ((pred_y == y).sum() / len(y)).item()

class GCN(torch.nn.Module):
    """Graph Convolutional Network"""
    def __init__(self, dim_in, dim_h, dim_out):
        super().__init__()
        self.gcn1 = GCNConv(dim_in, dim_h)
        self.gcn2 = GCNConv(dim_h, dim_out)

    def forward(self, x, edge_index):
        h = self.gcn1(x, edge_index)
        h = torch.relu(h)
        h = self.gcn2(h, edge_index)
        return F.log_softmax(h, dim=1)

```

```

def fit(self, data, epochs):
    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(self.parameters(), lr=0.01, weight_decay=5e-4)
    self.train()
    for epoch in range(epochs+1):
        optimizer.zero_grad()
        out = self(data.x, data.edge_index)
        loss = criterion(out[data.train_mask], data.y[data.train_mask])
        acc = accuracy(out[data.train_mask].argmax(dim=1), data.y[data.train_mask])
        loss.backward()
        optimizer.step()
        if (epoch % 20 == 0):
            val_loss = criterion(out[data.val_mask], data.y[data.val_mask])
            val_acc = accuracy(out[data.val_mask].argmax(dim=1), data.y[data.val_mask])
            print(f'Epoch {epoch:>3} | Train Loss: {loss:.3f} | '
                  'Train Acc: {acc*100:>5.2f}% | '
                  'Val Loss: {val_loss:.2f} | Val Acc: {val_acc*100:.2f}%')
            @torch.no_grad()
        def test(self, data):
            self.eval()
            out = self(data.x, data.edge_index)
            acc = accuracy(out.argmax(dim=1)[data.test_mask],
                           data.y[data.test_mask])
            return acc

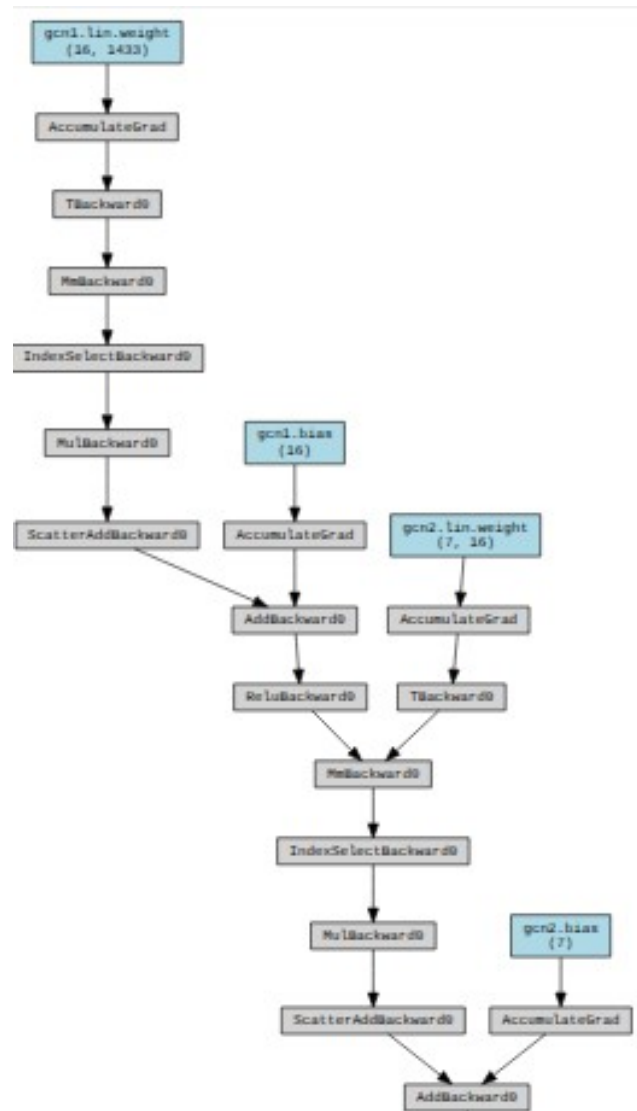
gcn = GCN(dataset.num_features, 16, dataset.num_classes)
print(gcn)
gcn.fit(data, epochs=100)

```

```

from torchviz import make_dot
out = gcn(data.x[:, :], data.edge_index[:, :])
dot = make_dot(out, params=dict(gcn.named_parameters()))
dot

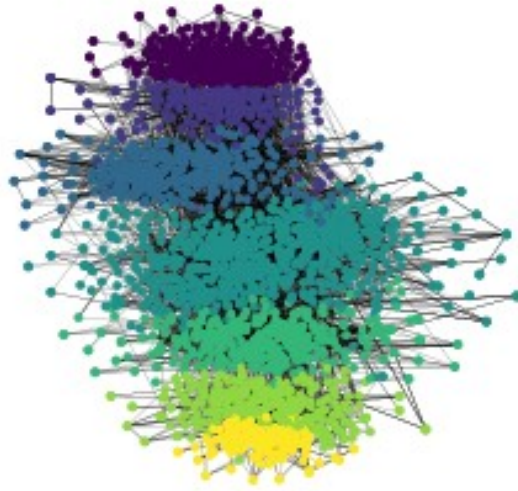
```



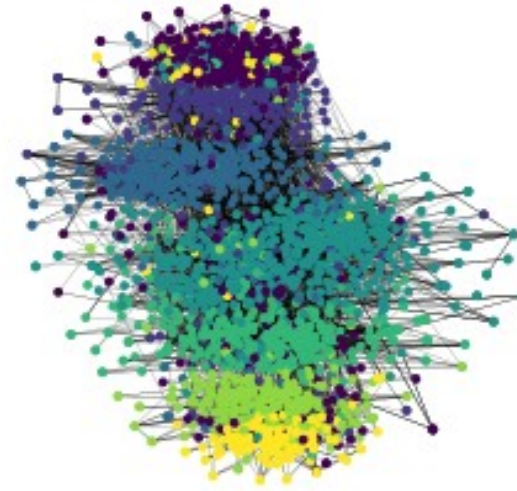
```
gcn.fit(data, epochs=100)  
yp=gcn(data.x,data.edge_index)
```

```
plt.figure(figsize=(7,3))  
plt.subplot(1,2,1)  
plt.title('labels')  
nx.draw(g,x5,node_color=data.y,node_size=3,width=0.1)  
plt.subplot(1,2,2)  
plt.title('predictions')  
nx.draw(g,x5,node_color=yp2.detach().numpy(),node_size=3,width=0.1)
```

labels



predictions



Оценка качества

```
from sklearn.metrics import classification_report  
print(classification_report(data.y,yp2.detach().numpy()))
```

	precision	recall	f1-score	support
0	0.67	0.76	0.71	351
1	0.73	0.88	0.79	217
2	0.88	0.92	0.90	418
3	0.92	0.75	0.83	818
4	0.80	0.85	0.82	426
5	0.81	0.78	0.80	298
6	0.70	0.82	0.75	180
accuracy			0.81	2708
macro avg	0.79	0.82	0.80	2708
weighted avg	0.82	0.81	0.81	2708

```
from sklearn.metrics import adjusted_rand_score,rand_score  
adjusted_rand_score(data.y,yp2.detach().numpy()),rand_score(data.y,yp2.detach().numpy())
```

```
(0.6128818901257318, np.float64(0.8903952169521657))
```

GraphSAGE

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT} \left(\mathbf{h}_v^{(l-1)}, \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right) \right) \right)$$

■ **How to write this as Message + Aggregation?**

■ **Message** is computed within the $\text{AGG}(\cdot)$

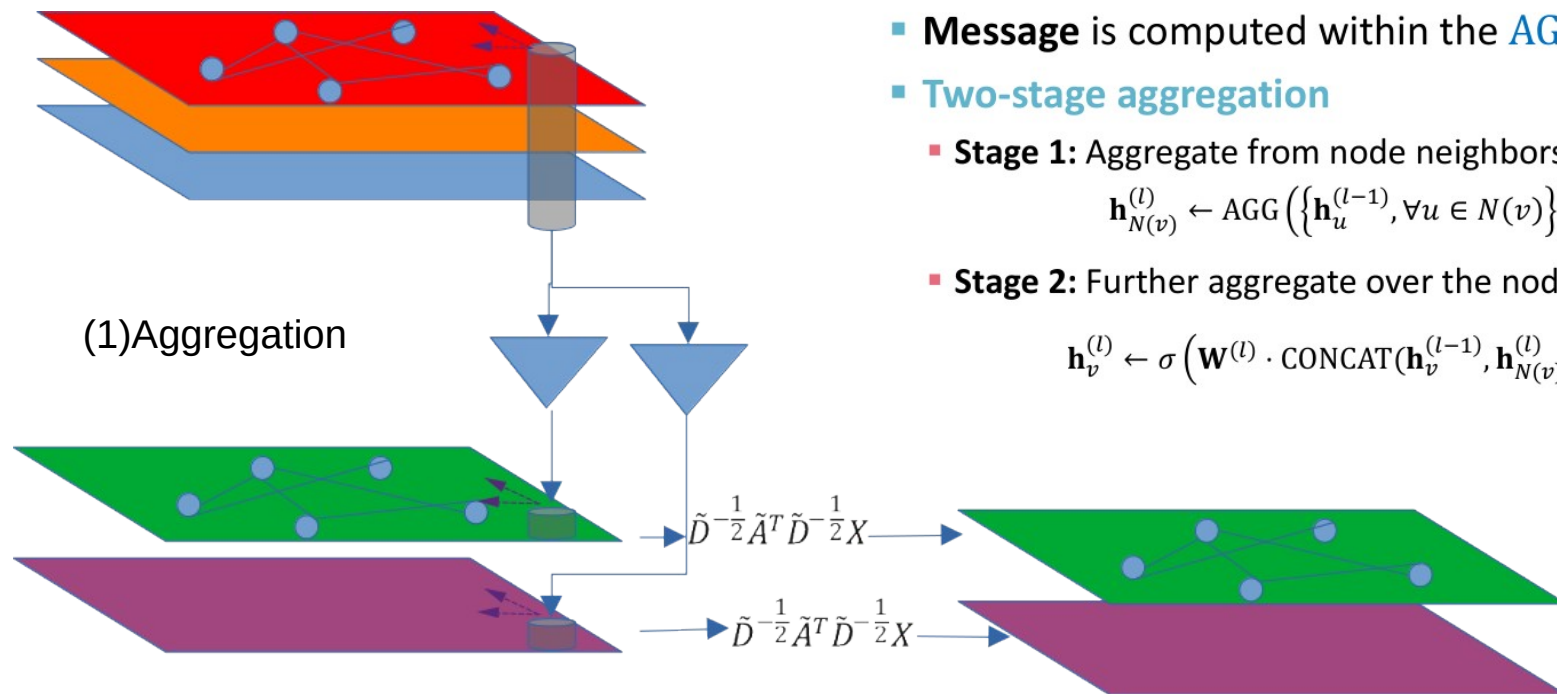
■ **Two-stage aggregation**

■ **Stage 1:** Aggregate from node neighbors

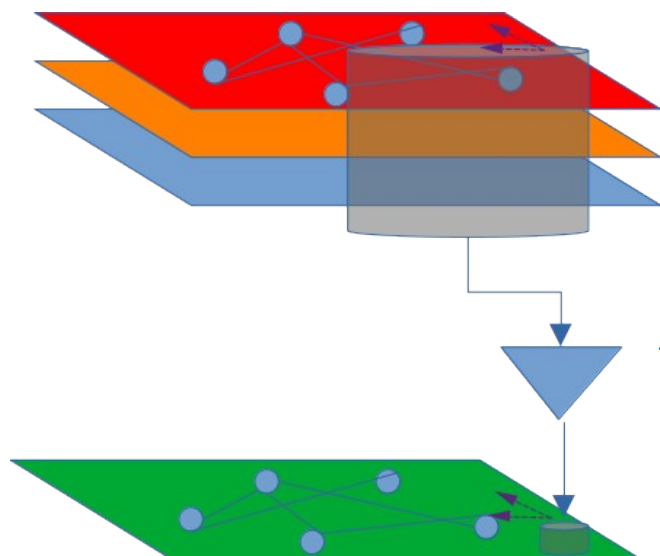
$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right)$$

■ **Stage 2:** Further aggregate over the node itself

$$\mathbf{h}_v^{(l)} \leftarrow \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)}) \right)$$



(2) Message transformation



- **Mean:** Take a weighted average of neighbors

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \quad \text{Aggregation} \quad \text{Message computation}$$

- **Pool:** Transform neighbor vectors and apply symmetric vector function Mean($\cdot$) or Max($\cdot$)

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\}) \quad \text{Aggregation} \quad \text{Message computation}$$

- **LSTM:** Apply LSTM to reshuffled of neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))]) \quad \text{Aggregation}$$

Then:

- **ℓ_2 Normalization:**

- **Optional:** Apply ℓ_2 normalization to $\mathbf{h}_v^{(l)}$ at every layer

- $\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\|\mathbf{h}_v^{(l)}\|_2} \quad \forall v \in V \text{ where } \|u\|_2 = \sqrt{\sum_i u_i^2} \text{ } (\ell_2\text{-norm})$

- Without ℓ_2 normalization, the embedding vectors have different scales (ℓ_2 -norm) for vectors
- In some cases (not always), normalization of embedding results in performance improvement
- After ℓ_2 normalization, all vectors will have the same ℓ_2 -norm

GAT

(Graph Attention Network)

■ (3) Graph Attention Networks

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \underbrace{\alpha_{vu}}_{\text{Attention weights}} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

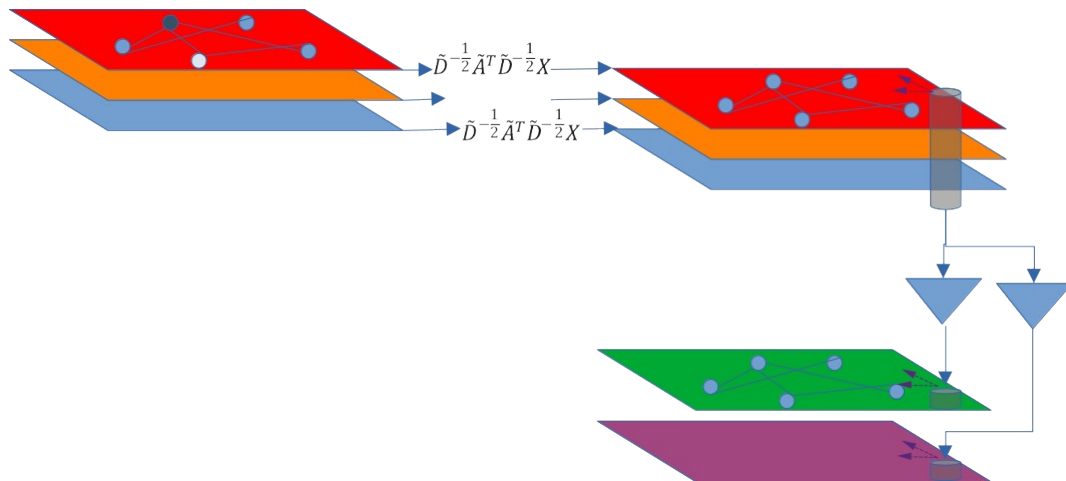
■ In GCN / GraphSAGE

- $\alpha_{vu} = \frac{1}{|N(v)|}$ is the **weighting factor (importance)** of node u 's message to node v
- $\Rightarrow \alpha_{vu}$ is defined **explicitly** based on the structural properties of the graph (node degree)
- $\Rightarrow$ All neighbors $u \in N(v)$ are equally important to node v

In GAT:

Not all node's neighbors are equally important

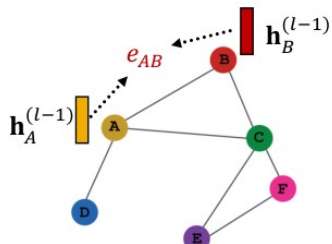
- **Attention** is inspired by cognitive attention.
- The **attention** α_{vu} focuses on the important parts of the input data and fades out the rest.
 - **Idea:** the NN should devote more computing power on that small but important part of the data.
 - Which part of the data is more important depends on the context and is learned through training.



- Let α_{vu} be computed as a byproduct of an **attention mechanism a** :
- (1) Let a compute **attention coefficients e_{vu}** across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_v^{(l-1)})$$

- e_{vu} indicates the importance of u 's message to node v



$$e_{AB} = a(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)})$$

- Normalize** e_{vu} into the **final attention weight α_{vu}**
- Use the **softmax** function, so that $\sum_{u \in N(v)} \alpha_{vu} = 1$:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

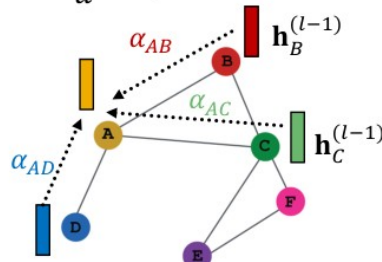
- Weighted sum** based on the **final attention weight**

α_{vu}

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

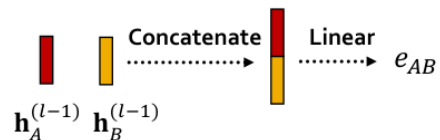
Weighted sum using $\alpha_{AB}, \alpha_{AC}, \alpha_{AD}$:

$$\mathbf{h}_A^{(l)} = \sigma(\alpha_{AB} \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} + \alpha_{AC} \mathbf{W}^{(l)} \mathbf{h}_C^{(l-1)} + \alpha_{AD} \mathbf{W}^{(l)} \mathbf{h}_D^{(l-1)})$$



■ What is the form of attention mechanism α ?

- The approach is agnostic to the choice of α
 - E.g., use a simple single-layer neural network
 - α have trainable parameters (weights in the Linear layer)

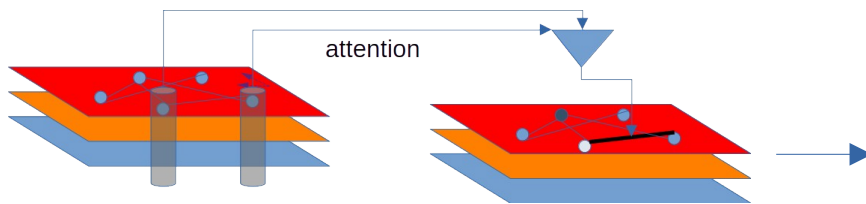


$$e_{AB} = a(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)})$$

$$= \text{Linear}(\text{Concat}(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)}))$$

■ Parameters of α are trained jointly:

- Learn the parameters together with weight matrices (i.e., other parameter of the neural net $\mathbf{W}^{(l)}$) in an end-to-end fashion



Attention adds additional weight to each link

Train everything end-to-end!

■ Multi-head attention: Stabilizes the learning process of attention mechanism

- Create **multiple attention scores** (each replica with a different set of parameters):

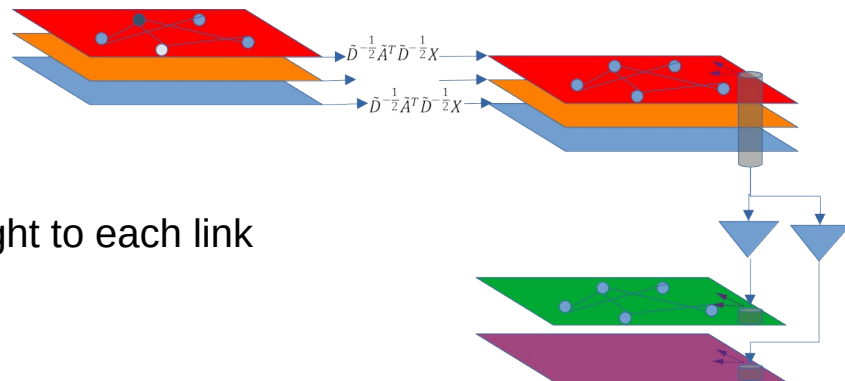
$$\mathbf{h}_v^{(l)}[1] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^1 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[2] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^2 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[3] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^3 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

■ Outputs are aggregated:

- By concatenation or summation
- $\mathbf{h}_v^{(l)} = \text{AGG}(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3])$



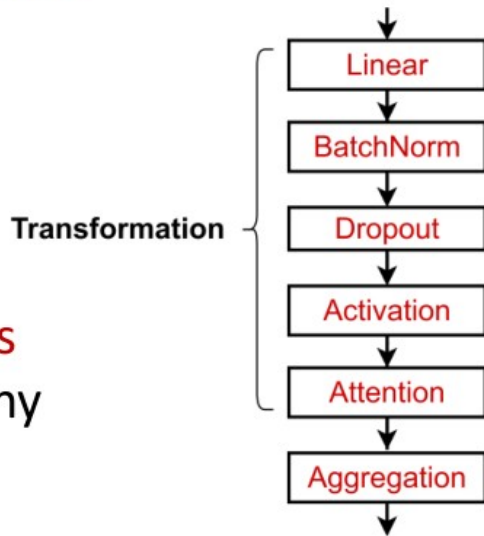
- **Key benefit:** Allows for (implicitly) specifying **different importance values (α_{vu}) to different neighbors**
- **Computationally efficient:**
 - Computation of attentional coefficients can be parallelized across all edges of the graph
 - Aggregation may be parallelized across all nodes
- **Storage efficient:**
 - Sparse matrix operations do not require more than $O(V + E)$ entries to be stored
 - **Fixed** number of parameters, irrespective of graph size
- **Localized:**
 - Only **attends over local network neighborhoods**
- **Inductive capability:**
 - It is a shared *edge-wise* mechanism
 - It does not depend on the global graph structure

Классические сверточные операции в графовых сетях

- In practice, these classic GNN layers are a great starting point

- We can often get better performance by considering a general GNN layer design
- Concretely, we can include modern deep learning modules that proved to be useful in many domains

A suggested GNN Layer



Батч-нормализация

- **Goal:** Stabilize neural networks training
- **Idea:** Given a batch of inputs (node embeddings)
 - Re-center the node embeddings into zero mean
 - Re-scale the variance into unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times D}$
 N node embeddings

Trainable Parameters:
 $\gamma, \beta \in \mathbb{R}^D$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$
Normalized node embeddings

Step 1:
Compute the mean and variance over N embeddings

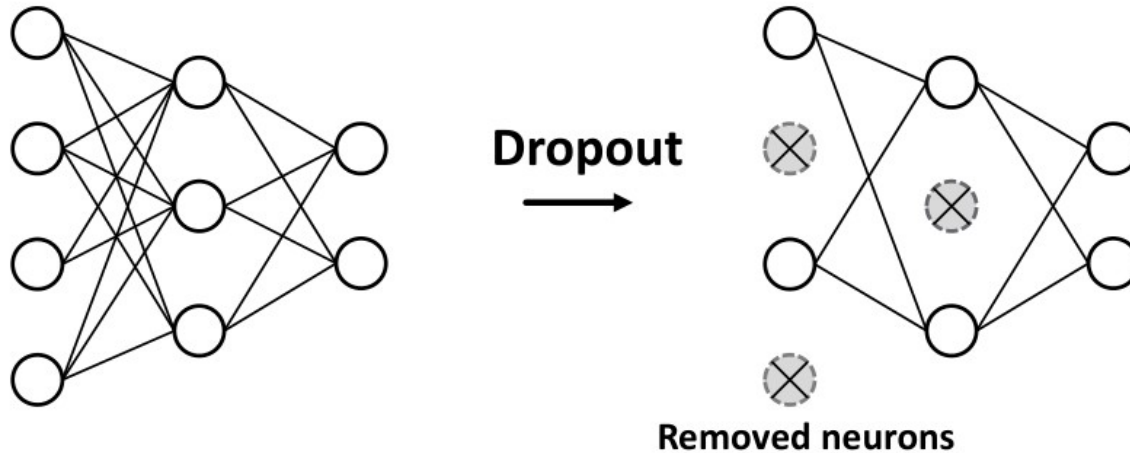
$$\mu_j = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{i,j} - \mu_j)^2$$

Step 2:
Normalize the feature using computed mean and variance

$$\hat{\mathbf{x}}_{i,j} = \frac{\mathbf{x}_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
$$\mathbf{y}_{i,j} = \gamma_j \hat{\mathbf{x}}_{i,j} + \beta_j$$

Drop-out

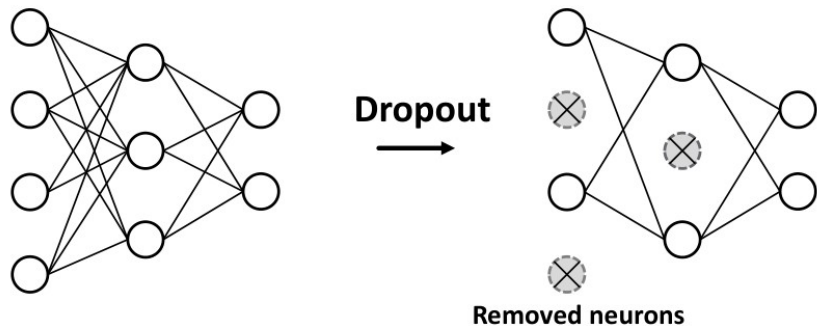
- **Goal:** Regularize a neural net to prevent overfitting.
- **Idea:**
 - **During training:** with some probability p , randomly set neurons to zero (turn off)
 - **During testing:** Use all the neurons for computation



Drop-out

In NN

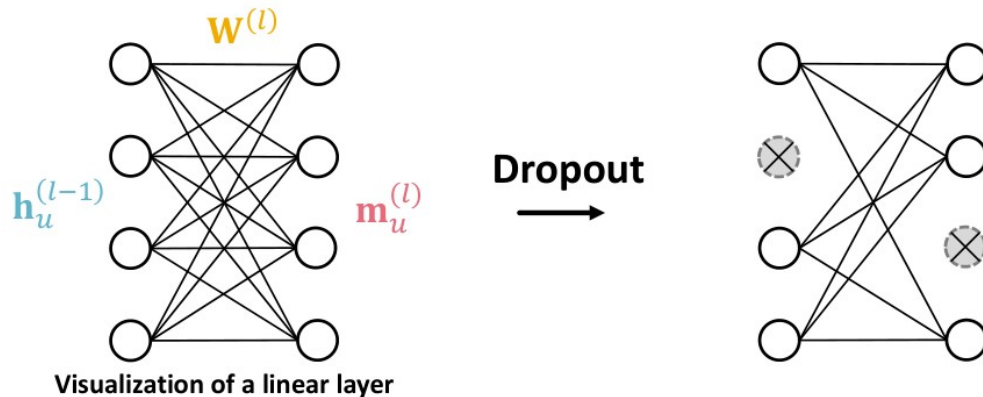
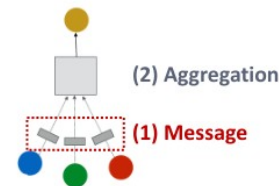
- **Goal:** Regularize a neural net to prevent overfitting.
- **Idea:**
 - **During training:** with some probability p , randomly set neurons to zero (turn off)
 - **During testing:** Use all the neurons for computation



In GNN

- In GNN, Dropout is applied to **the linear layer in the message function**

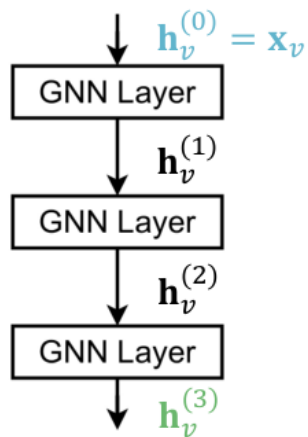
- A simple message function with linear layer: $m_u^{(l)} = W^{(l)} h_u^{(l-1)}$



Randomly skip neighbours during message transformation

■ How to construct a Graph Neural Network?

- **The standard way:** Stack GNN layers sequentially
- **Input:** Initial raw node feature $\mathbf{x}_v$
- **Output:** Node embeddings $\mathbf{h}_v^{(L)}$ after L GNN layers

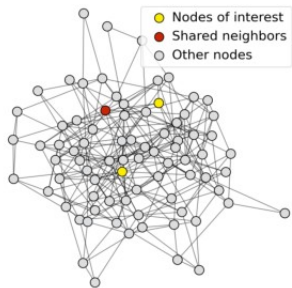


■ The Issue of stacking many GNN layers

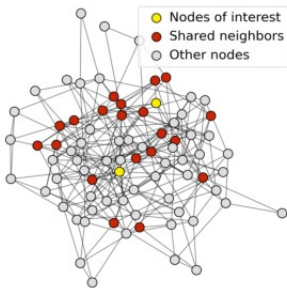
- GNN suffers from **the over-smoothing problem**
- **The over-smoothing problem:** all the node embeddings converge to the same value
 - This is bad because we **want to use node embeddings to differentiate nodes**
- **Why does the over-smoothing problem happen?**

- We can explain over-smoothing via the notion of receptive field
 - We knew the embedding of a node is determined by its receptive field
 - If two nodes have highly-overlapped receptive fields, then their embeddings are highly similar
 - Stack many GNN layers → nodes will have highly-overlapped receptive fields → node embeddings will be highly similar → suffer from the over-smoothing problem
- Next: how do we overcome over-smoothing problem?
 - Receptive field overlap for two nodes
 - The shared neighbors quickly grows when we increase the number of hops (num of GNN layers)
- What do we learn from the over-smoothing problem?
 - Lesson 1: Be cautious when adding GNN layers
 - Unlike neural networks in other domains (CNN for image classification), adding more GNN layers do not always help
 - Step 1: Analyze the necessary receptive field to solve your problem. E.g., by computing the diameter of the graph
 - Step 2: Set number of GNN layers L to be a bit more than the receptive field we like. Do not set L to be unnecessarily large!
 - Question: How to enhance the expressive power of a GNN, if the number of GNN layers is small?

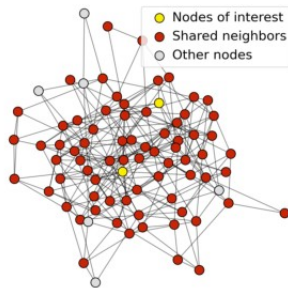
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



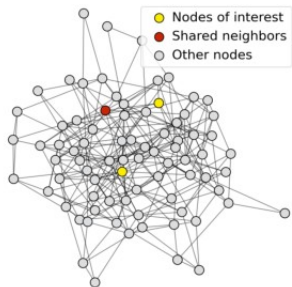
3-hop neighbor overlap
Almost all the nodes!



- We can explain over-smoothing via the notion of receptive field
 - We knew the embedding of a node is determined by its receptive field
 - If two nodes have highly-overlapped receptive fields, then their embeddings are highly similar
 - Stack many GNN layers → nodes will have highly-overlapped receptive fields → node embeddings will be highly similar → suffer from the over-smoothing problem
- Next: how do we overcome over-smoothing problem?
 - Receptive field overlap for two nodes
 - The shared neighbors quickly grows when we increase the number of hops (num of GNN layers)
- What do we learn from the over-smoothing problem?
 - Lesson 1: Be cautious when adding GNN layers
 - Unlike neural networks in other domains (CNN for image classification), adding more GNN layers do not always help
 - Step 1: Analyze the necessary receptive field to solve your problem. E.g., by computing the diameter of the graph
 - Step 2: Set number of GNN layers L to be a bit more than the receptive field we like. Do not set L to be unnecessarily large!
 - Question: How to enhance the expressive power of a GNN, if the number of GNN layers is small?

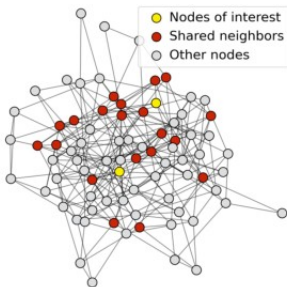
1-hop neighbor overlap

Only 1 node



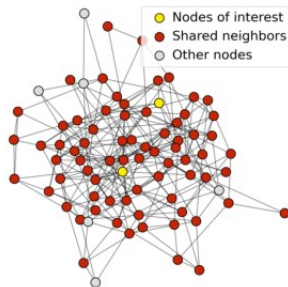
2-hop neighbor overlap

About 20 nodes



3-hop neighbor overlap

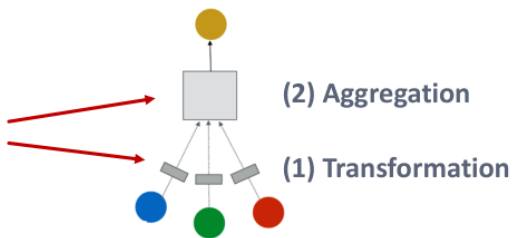
Almost all the nodes!



- **How to make a shallow GNN more expressive?**
- **Solution 1:** Increase the expressive power **within each GNN layer**

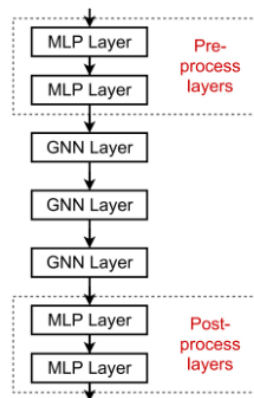
- In our previous examples, each transformation or aggregation function only include one linear layer
- We can **make aggregation / transformation become a deep neural network!**

If needed, each box could include a **3-layer MLP**



- **How to make a shallow GNN more expressive?**
- **Solution 2:** Add layers that do not pass messages

- A GNN does not necessarily only contain GNN layers
 - E.g., we can add **MLP layers** (applied to each node) before and after GNN layers, as **pre-process layers** and **post-process layers**



Pre-processing layers: Important when encoding node features is necessary.
E.g., when nodes represent images/text

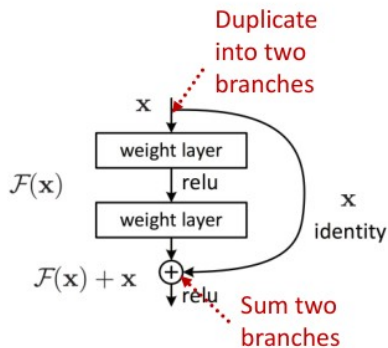
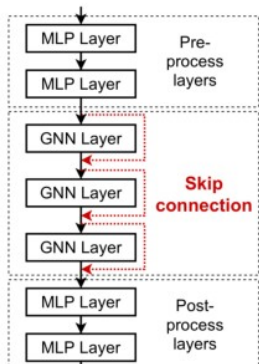
Post-processing layers: Important when reasoning / transformation over node embeddings are needed
E.g., graph classification, knowledge graphs

In practice, adding these layers works great!

- What if my problem still requires many GNN layers?

- Lesson 2: Add skip connections in GNNs**

- Observation from over-smoothing:** Node embeddings in earlier GNN layers can sometimes better differentiate nodes
- Solution:** We can increase the impact of earlier layers on the final node embeddings, **by adding shortcuts in GNN**



Idea of skip connections:

Before adding shortcuts:

$$F(\mathbf{x})$$

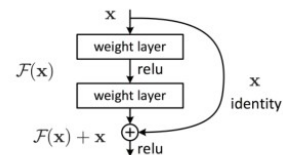
After adding shortcuts:

$$F(\mathbf{x}) + \mathbf{x}$$

- A standard GCN layer**

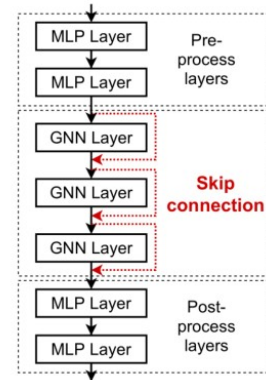
$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \right)$$

This is our $F(\mathbf{x})$



- A GCN layer with skip connection**

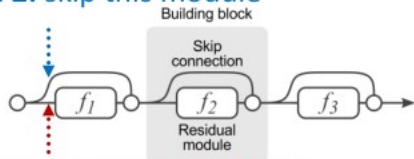
$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} + \mathbf{h}_v^{(l-1)} \right)$$



■ Why do skip connections work?

- **Intuition:** Skip connections create **a mixture of models**
- N skip connections $\rightarrow 2^N$ possible paths
- Each path could have up to N modules
- We automatically get **a mixture of shallow GNNs and deep GNNs**

Path 2: skip this module

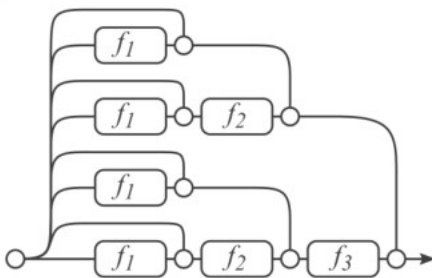


Path 1: include this module

(a) Conventional 3-block residual network

All the possible paths:

$$2 * 2 * 2 = 2^3 = 8$$



(b) Unraveled view of (a)

- **Other options:** Directly skip to the last layer
 - The final layer directly **aggregates from all the node embeddings** in the previous layers

